Week 6 Seminar: Regression and Clustering in Python

Type along. Every block on these slides runs as it stands.

Acknowledgement of Country

We respectfully acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne's Australian campuses are located in Melbourne's east and outer-east, and pay our respect to their Elders past, present and emerging.

We are honoured to recognise our connection to Wurundjeri Country, history, culture, and spirituality through these locations, and strive to ensure that we operate in a manner that respects and honours the Elders and Ancestors of these lands.

We also respectfully acknowledge Swinburne's Aboriginal and Torres Strait Islander staff, students, alumni, partners and visitors.

We also acknowledge and respect the Traditional Owners of lands across Australia, their Elders, Ancestors, cultures, and heritage, and recognise the continuing sovereignties of all Aboriginal and Torres Strait Islander Nations.



What you will do today

1. **Explore a dataset in code, not by staring at it**
2. **Build a feature that did not exist, and check whether it helps**

This is the skill Assignment 2 is built around

3. **Fit and read a linear regression, then measure it four ways**
4. **Use cross-validation, and find out how lucky your first number was**
5. **Cluster the same data without labels, then describe what you found**

Open a notebook now. Everything today runs in pandas, scikit-learn and matplotlib, and nothing downloads more than 100 KB.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Setup and load

Run this first. If it fails, put your hand up.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import sklearn

URL = ("https://raw.githubusercontent.com/plotly/"
       "datasets/master/winequality-red.csv")
wine = pd.read_csv(URL)

print(sklearn.__version__)
print(wine.shape)
```

output

```
1.8.0
(1599, 12)
```

If an import fails

pip install pandas scikit-learn matplotlib (put ! in front inside a notebook)

Version matters this year: code written for scikit-learn 1.1 or earlier may call load_boston, which was removed in 1.2 and now raises ImportError.

SWINBURNE UNIVERSITY OF TECHNOLOGY

One thing about notebooks that trips everyone up

Why some lines show output and others silently do nothing

```
# in a NOTEBOOK cell

wine.shape
wine.duplicated().sum()

# only the LAST line displays.
# the first one vanishes.
```

output

```
240

where did (1599, 12) go?
```

```
# and in a .py SCRIPT

wine.shape
wine.duplicated().sum()

# nothing at all is displayed.
```

output

```
no output whatsoever
```

Wrap anything you want to see in print(). It works in both, it shows every line rather than the last, and your code still **works when you move it out of the notebook and into your Assignment 3 backend.**

SWINBURNE UNIVERSITY OF TECHNOLOGY

Explore it in code

Five commands, and you avoid most of the later mistakes

```
print(wine.head(3))
print(wine.dtypes.unique())
print(wine.isnull().sum().sum())
print(wine.duplicated().sum())
print(wine.describe().loc[['min', 'max']])
```

output

```
all columns float64 or int64
0         no missing values
240       exact repeat rows

alcohol     8.40 to 14.90
density     0.990 to 1.004
total SO2   6.00 to 289.00
```

Two things worth stopping on

240 duplicate rows out of 1,599. They may be genuinely identical wines, or the same wine entered twice. Decide, say why in your report, and do it BEFORE you split, or the same row lands in both train and test.

Look at those ranges: total sulfur dioxide spans 283 while density spans 0.014. That gap will matter enormously when we get to clustering.

wine.describe() gives you eight rows. Slicing it with .loc[['min','max']] keeps the output small enough to actually read, which matters when you have thirty columns.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Ask the data a question

groupby is the single most useful thing pandas does

```
print(wine.groupby('quality').alcohol.mean().round(2))

print(wine.groupby('quality').size())

# and the same thing as a chart
wine.groupby('quality').alcohol.mean().plot(
    kind='bar', ylabel='mean alcohol %')
plt.show()
```

output

```
quality
3      9.96
4     10.27
5      9.90
6     10.63
7     11.47
8     12.09
```

There is your first real finding, and it took one line: better wines have more alcohol, from 9.9 percent at quality 5 up to 12.1 at quality 8. But look at quality 3 and 4. They break the pattern, and there are only 10 and 53 of them. Small groups are noisy, and a chart that hides the group sizes will mislead you. Always print size() alongside the means.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Look at the target before you model it

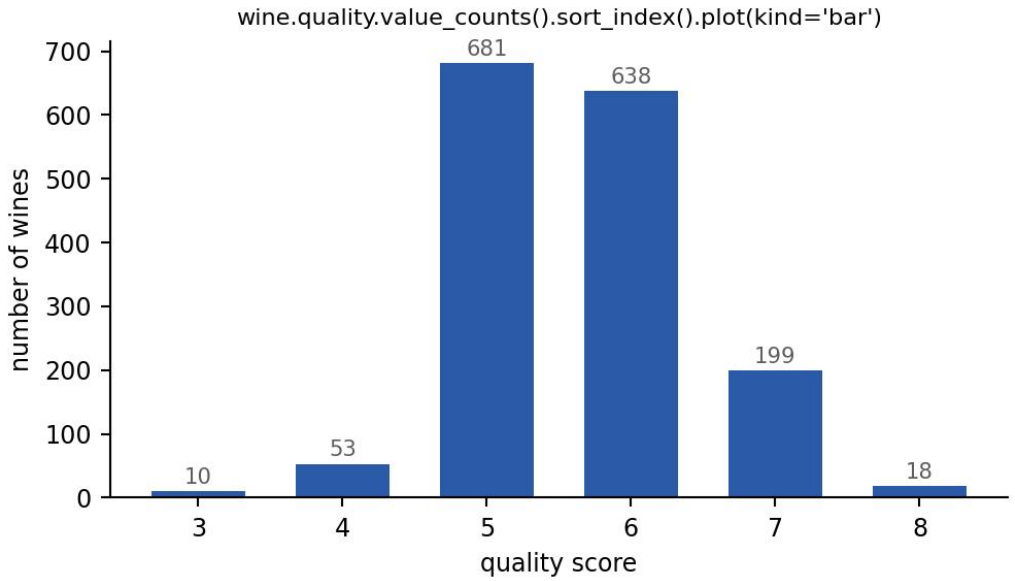
```
print(wine.quality.value_counts().sort_index())

wine.quality.hist(bins=6)
plt.xlabel('quality'); plt.ylabel('count')
plt.show()
```

Most wines score 5 or 6. Only 18 score 8, only 10 score 3.

For regression that means a model can do reasonably well while never predicting an extreme at all.

Next week, when we split this into good and bad, we choose the cut point knowing this shape.



Part 1

Building a feature that was not there

The skill Assignment 2 is actually about.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Make a feature and test whether it earns its place

Two ratios, invented from columns that already exist

```
wine['acid_ratio'] = wine['fixed acidity'] / wine['volatile acidity']
wine['so2_ratio']  = wine['free sulfur dioxide'] / wine['total sulfur dioxide']

for col in ['alcohol', 'acid_ratio', 'so2_ratio', 'pH']:
    print(f'{col:<12} {wine[col].corr(wine.quality):+.3f}')
```

output

```
alcohol      +0.476
acid_ratio   +0.343
so2_ratio    +0.194
pH           -0.058
```

Read that carefully

acid_ratio did not exist a moment ago, and it correlates at +0.343, stronger than any raw column except alcohol.

You made information out of arithmetic.

This is exactly what you will do in Assignment 2, except your raw input is a URL or a log line rather than two numeric columns. The move is the same: combine what you have into something that separates the classes better.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Part 2

Regression

Predict the quality score from the chemistry.

SWINBURNE UNIVERSITY OF TECHNOLOGY

X, y, and the split

Three lines, three traps

```
from sklearn.model_selection import train_test_split

X = wine[["alcohol"]]        # DataFrame: note the double brackets
y = wine["quality"]          # Series: single brackets

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
```

double brackets	sklearn wants X two-dimensional. A Series raises: Expected 2D array, got 1D array
test_size=0.3	hold back 30 percent. Smaller means a noisier estimate of how good you are.
random_state=42	fixes the shuffle. Leave it out and every number you report moves each run.
the return order	X_train, X_test, y_train, y_test. Swap the middle two and it still runs, wrongly.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Fit, then read every attribute it gives you

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

print('slope     ', model.coef_)
print('intercept ', model.intercept_)
print('score     ', model.score(X_test, y_test))

# predict a wine we invented
print(model.predict([[12.0]]))
```

output

```
slope      [0.3770]
intercept  1.6947
score      0.1848

[6.22]

a 12% wine scores 6.22
```

coef_ and intercept_ are b1 and b0 from the lecture. The whole model is those two numbers.

The trailing underscore is a scikit-learn convention for attributes created by fit. Read coef_ first and you get AttributeError.

.score() on a regressor returns R squared, so it is the same 0.185 you get from r2_score. On a classifier it returns accuracy instead.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Measure it four ways, and plot it

```
from sklearn.metrics import (mean_squared_error,
                             mean_absolute_error, r2_score)

y_pred = model.predict(X_test)

print('MSE ', mean_squared_error(y_test, y_pred))
print('RMSE', mean_squared_error(y_test, y_pred) ** 0.5)
print('MAE ', mean_absolute_error(y_test, y_pred))
print('R2  ', r2_score(y_test, y_pred))
```

output

```
MSE    0.517
RMSE   0.719
MAE    0.578
R2     0.185
```

```
plt.scatter(y_test, y_pred, s=10, alpha=0.35)
plt.plot([3, 8], [3, 8], 'r--')                  # where a perfect model would sit
plt.xlabel('actual'); plt.ylabel('predicted')
```

MAE 0.578 is below RMSE 0.719. That gap exists because RMSE squares the errors, so a few badly missed wines pull it up. When the two are far apart, go and look at the worst predictions individually.

SWINBURNE UNIVERSITY OF TECHNOLOGY

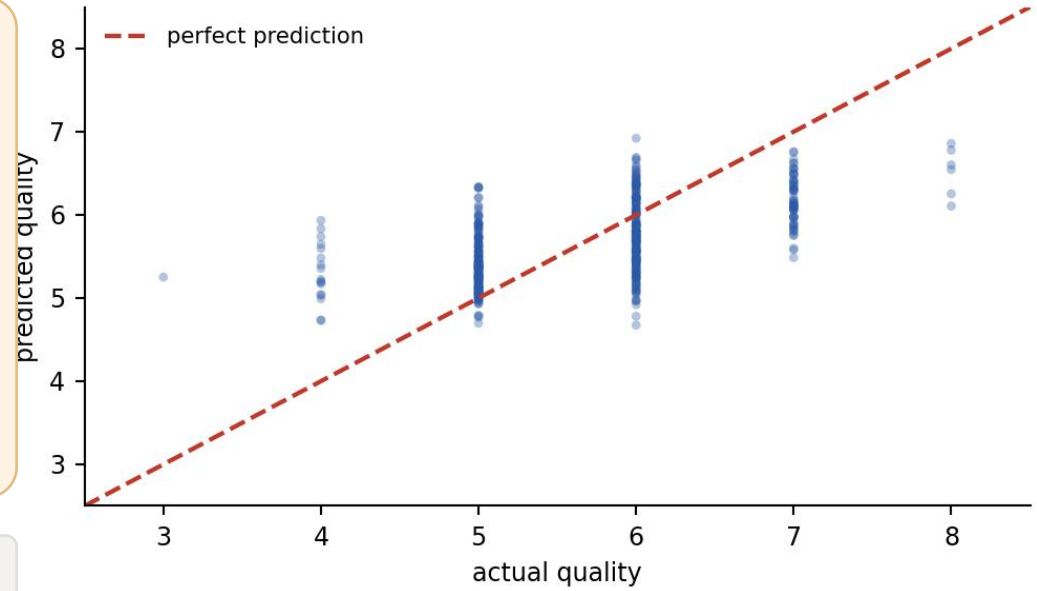
What that plot tells you that no metric did

Look at the vertical spread, and at the range.

Real wines score anywhere from 3 to 8. Almost every prediction lands between 5 and 6.5.

The model has learned to play safe near the middle, because guessing the average is rarely badly wrong.

R squared 0.185 does not say that. RMSE 0.719 does not say that. Only the picture does.




```
worst = (y_test - y_pred).abs().nlargest(5)
print(wine.loc[worst.index, ['alcohol', 'quality']])
# go and read the five wines it got most wrong
```

SWINBURNE UNIVERSITY OF TECHNOLOGY

All eleven features, then a reality check

One number from one split is not evidence

```
from sklearn.model_selection import cross_val_score

X = wine.drop(columns=["quality", "acid_ratio", "so2_ratio"])

scores = cross_val_score(LinearRegression(), X, y, cv=5, scoring='r2')
print(scores.round(3))
print(f'mean {scores.mean():.3f}   std {scores.std():.3f}')
```

output

```
[0.132  0.319  0.350  0.369  0.281]

mean 0.290   std 0.084
```

Why this matters

A single split gave 0.351. Five splits give anything from 0.132 to 0.369.

cv=5 splits the data into five parts, trains on four and tests on the fifth, five times over. It costs five times the compute and buys you an honest number.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Part 3

Clustering

Same data, no labels, a different question.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Scale first, and see what happens if you do not

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

X = wine.drop(columns=["quality"])       # labels dropped

raw = KMeans(3, n_init=10, random_state=42).fit_predict(X)

X_scaled = StandardScaler().fit_transform(X)
scaled = KMeans(3, n_init=10, random_state=42).fit_predict(X_scaled)

print(np.bincount(raw))
print(np.bincount(scaled))
```

output

```
without scaling
[ 763  649  187]

with scaling
[ 722  502  375]

different groups entirely
```

Unscaled, K-means is really clustering on total sulfur dioxide alone, because it spans 283 while density spans 0.014.

Ten of your eleven features had almost no say. Run both versions once so you have seen it happen.

SWINBURNE UNIVERSITY OF TECHNOLOGY

K-means, and what each argument decides

```
km = KMeans(n_clusters=3, n_init=10, random_state=42)
labels = km.fit_predict(X_scaled)

print(km.inertia_, np.bincount(labels))
```

`n_clusters=3`	this is k. You choose it, and the next slide is about how.
`n_init=10`	ten runs from different random starts, keeping the best. Cheap insurance.
`random_state=42`	makes those starts repeatable, so your report matches your notebook.
`fit_predict`	fits, then returns a cluster number per row. Same as fit then .labels_
`inertia_`	total squared distance to the assigned centroid. Lower is tighter, not better.

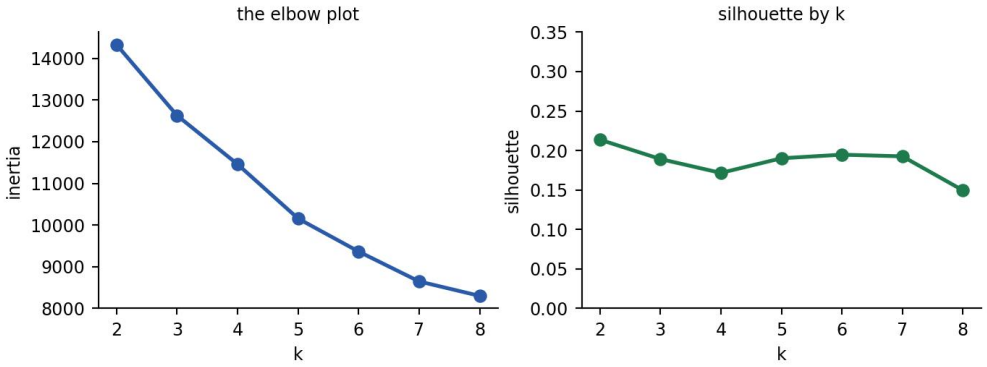
Print np.bincount(labels) every single time. A cluster holding a handful of rows means K-means locked onto outliers, and the fix is to transform your features, not to change k.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Choosing k, both measures in one loop

```
from sklearn.metrics import silhouette_score

for k in range(2, 9):
    km  = KMeans(k, n_init=10, random_state=42)
    lab = km.fit_predict(X_scaled)
    print(k,
          round(km.inertia_),
          round(silhouette_score(X_scaled, lab), 3))
```



Read this honestly.

Inertia falls smoothly with no clear elbow, and every silhouette sits near 0.2.

That is the data saying these wines do not form separated groups. Reporting it, with numbers, beats picking k=3 and pretending.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Describe every cluster

The part Assignment 2 marks

```
wine["cluster"] = labels
overall, spread = X.mean(), X.std()

for c in sorted(wine.cluster.unique()):
    g = wine[wine.cluster == c]
    z = (g[X.columns].mean() - overall) / spread      # standard deviations from normal
    high, low = z.nlargest(2), z.nsmallest(2)
    print(f'cluster {c}  n={len(g)}')
    print('   more:', high.round(2).to_dict())
    print('   less:', low.round(2).to_dict())
```

output

```
cluster 0  n=722    more: pH +0.6, volatile acidity +0.5      less: citric acid -0.8
cluster 1  n=502    more: citric acid +1.0, fixed acidity +1.0   less: pH -1.0
cluster 2  n=375    more: total SO2 +1.3, free SO2 +1.1        less: sulphates -0.2
```

Dividing by the standard deviation is the whole trick. Without it total sulfur dioxide wins every comparison because it is measured in tens while pH is measured in units. Now the clusters have names: cluster 1 is the high-acid group.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Errors you will hit today

All four are normal

`ValueError: Expected 2D array, got 1D array instead`

You passed a Series where a DataFrame was wanted. Add the second pair of brackets.

`AttributeError: LinearRegression object has no attribute coef_`

You read coef_ before calling fit. Underscore attributes appear afterwards.

`ConvergenceWarning: Number of distinct clusters found smaller than n_clusters`

You asked for more clusters than the data supports. Lower k.

`ImportError: cannot import name load_boston`

Removed in scikit-learn 1.2. The tutorial you are following is out of date.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Before you leave

Invent two more features and test them

Anything you can justify: a ratio, a difference, a product. Print the correlation with quality. Keep the ones above 0.2 and say why you dropped the rest.

Cluster only the wines scoring 6 or above

Every cluster then sits in one quality band, so the label tells you nothing and you must describe by content. That is exactly the Assignment 2 setup.

Break it on purpose, twice

Run K-means without scaling, then drop random_state and run three times. Explain each result in one sentence.

Next week: the same eleven columns, but the question becomes good or bad, and we finish with saving a trained model to disk, which is what Assignment 3 loads.

SWIN BUR NE
SWINBURNE UNIVERSITY OF TECHNOLOGY